Problem-1)

X-16 Processor

We have to design the binary format of the instructions, Instructions are written in strings of 0s and 1's.

We are given 16 different operations: ADD, SUB, AND, OR, XOR, INC, DEC, MOV, LDI, LD, ST, BEQ, BNE, JMP, RET, NOP.

Each operations perform different functions and we have to represent them using diff codes. (opcodes)

- Opcode: It basically identifies which type of operation the processor has to perform.

Therefore Minimum opcode width required;

16 operation,

One bit can represent 2^n diff patterns,

$2^n = 16$

$n = \log_2[16] = 4$

Minimum opcode width = 4 bits

- Instruction code tells us what operation it is going to perform, what register its gonna use and which address memory it will use to store data.

No. of bits to represent Register field

R_0 to R_{15} ⇒ 16 diff registers.

∴ 4 bits are needed, $2^n \geq 16$ ($n \geq 4$)

(minimum)

Register width is 16 bit itself

Register field width = 4 bits.

No. of bits for LDI immediate field

given range : -8 to $+7$

$-8, -7, -6 \ldots 0, 1, 2, \ldots 7$

(16 diff values)

They are signed values, therefore they will be stored in 2's complement form.

$\Rightarrow \quad 2^n \geq 16$

$n \geq 4$

4 bits required to rep. all diff LDI immediate

Example

$-5 \rightarrow 1011$

2's comp of 5

0101

1's C 1010

+1

1011

No. of bits for Memory offset field

Range $\Rightarrow$ -8 to $+7$

Therefore it is same as LDI immediate

4 bits (2's complement form)

Note:- The address rule which we are expected to satisfy is that the memory address must be between 0 to 255 and must be even.

Ex: Ld : Rd, [Rb + off] $\Rightarrow$

Address = 20 + (−3) = 17 (odd)

It is invalid for our design because here memory is byte addressed, our data values are 16 bits so each value occupies two bits. bytes.

As stated, aligning 16 bit values to even address is standard rule.

Ex: Suppose we want to store three 16 bit values A, B & C,

Address	Hold by
0	A
1	A
2	B
3	B
4	C
5	C

{ Each new value start at even address (0,2,4) and the pairs never overlap for a valid memory addressing }

* Branch displacement

Two conditional operations are given:

BEQ: Rs1, RS2, disp (Branch if registers are equal)

BNE: Rs1, RS2, disp (Branch if registers are not equal)

The given range is −16 to +15 bytes

It contains 32 distinct values.

Therefore we need 5 ($2^n = 32$, $n = 5$) bits for the signed displacement (represented in form of 2's complement)

* JMP Target

It uses an absolute byte address,

0 to 255

0 to $(2^8 - 1)$ ⇒ 256 possible addresses.

$2^n \geq 256$ ⇒ 8 bits to represent the target address.

Example:

12 ⇒ 00001100

36 ⇒ 00100100

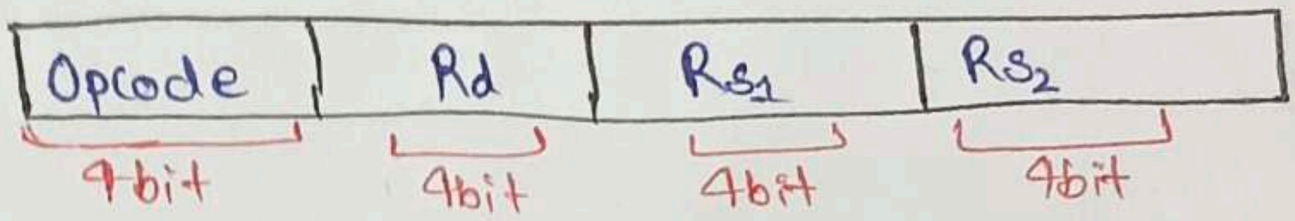
Choosing Instruction Length

We will use above found results to decide the instruction length.

[every instruction must have the same total length, even when different instructions use different operands.]

Ex① Three register instruction (Add : Rd, Rs1, Rs2)

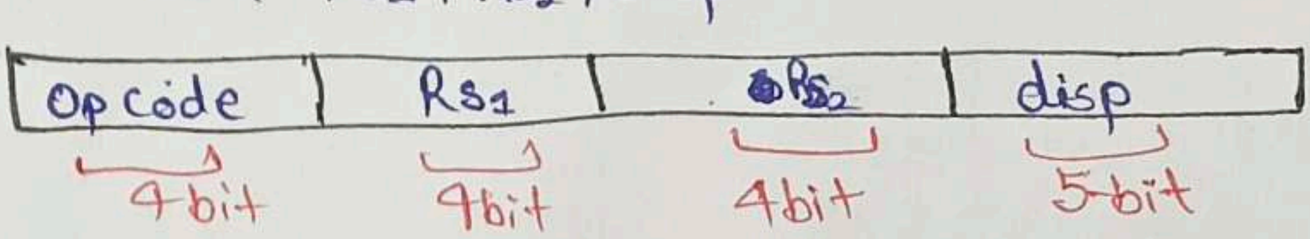
Reg → 4 bits each
opcode → 4 bits min required to represent opcode.



Total required bits = 4+4+4+4 = 16 bits minimum

Ex② Conditional branch

BEQ Rs1, Rs2, disp



Minimum bit required = 4+4+4+5 = 17 bits

↳ This will not fit in 16 bit instruction length

Therefore, 24bits is straight forward choice for our design. But using 24 bits will left some unused bits (padding bits)

↳ uses of unused bits ??

"unused" ⇒ they don't do anything

They pad to make every instruction look 24 bits long.

- They must be zero. (Keeping them zero will reserve them other than confusing it with any other code)

↳ unused bits matters for decoding?

The alignment changes with 24bits.

If decoder looked at all 24bits and compare them to one exact pattern, a stray bit would make it fail.

Designing the Instruction formats

We have to make a design that will be easy to decode.

↳ For that reason, we have to put the opcode at the same bit position for every format.

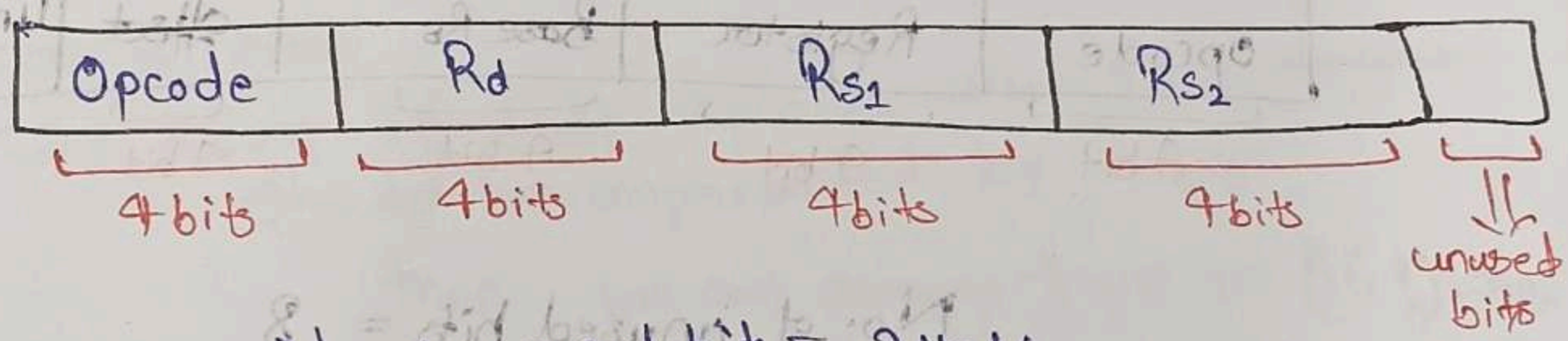
↳ The decoder should always read the opcode first and it should ignore the unused bits at first, by looking at the opcode only it will deduce decide which operation to perform and what format to use for the remaining bits.

For Example:- Formats

Three Register Instruction

It will be used in instructions ADD, SUB, AND, OR, XOR

If our opcode is represented by 4 bits, each register requires 4 bit to represent itself.



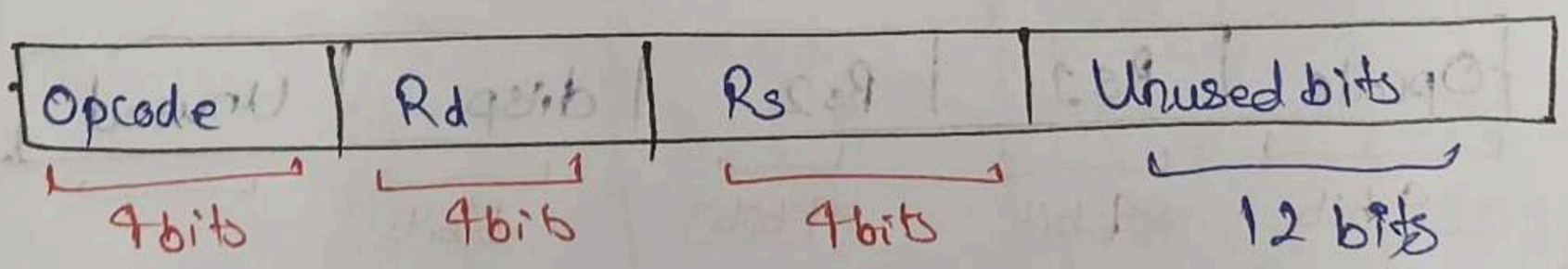
No. of unused bits = 24 - 16
= 8

We will set unused bits to 0, so that it will be easier to decode our instruction.

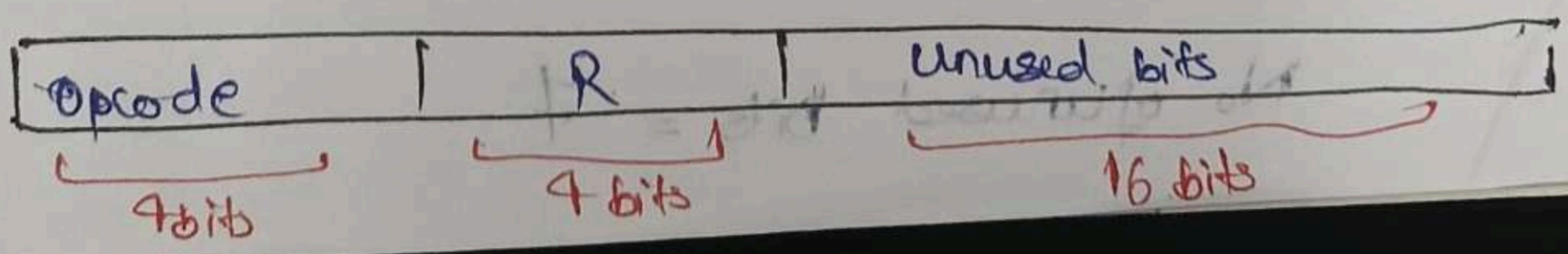
② Example2) Two or One Register Instruction

used in MOV, INC, DEC, RET

MOV

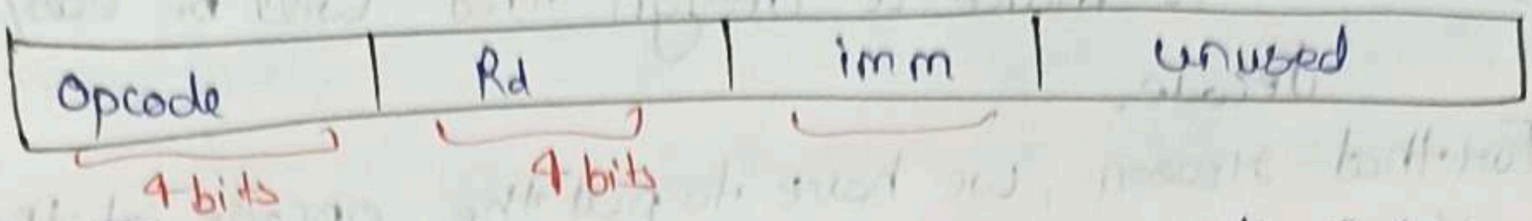


INC/DEC



③ ~~Eg:~~ Immediate Instruction (LDI)

Fields involved are opcode, Rd, imm, unused



To represent an LDI imm, we need 4 bits (as calculated earlier)

No: of unused bits = 24 - (4+4+4)
= 12 bits

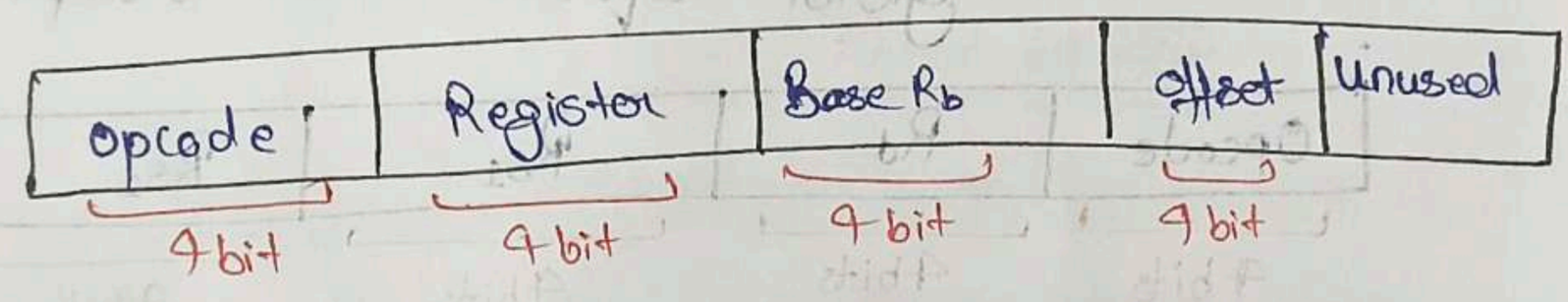
* The total length of all the instructions is only 24 bits.
(It cannot be less or more)

④ ~~Eg:~~ Memory Instructions

Type of instructions used: LD, ST

LD: Rd, [Rb + off]

ST: Rs, [Rb + off]

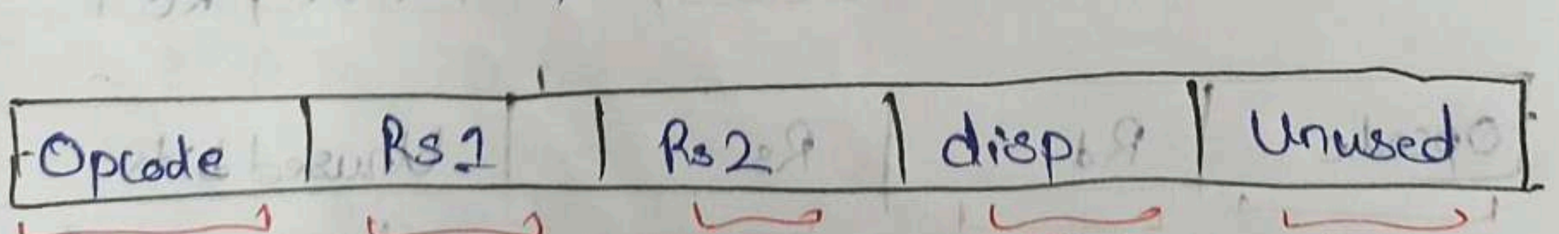


No. of unused bits = 8

⑤ ~~Eg:~~ Conditional Branches

BEQ: Rs1, Rs2, disp
BNE: Rs1, Rs2, disp

It uses a branch displacement operation which needs 5 bits for its representation.



No. of unused bits = 7

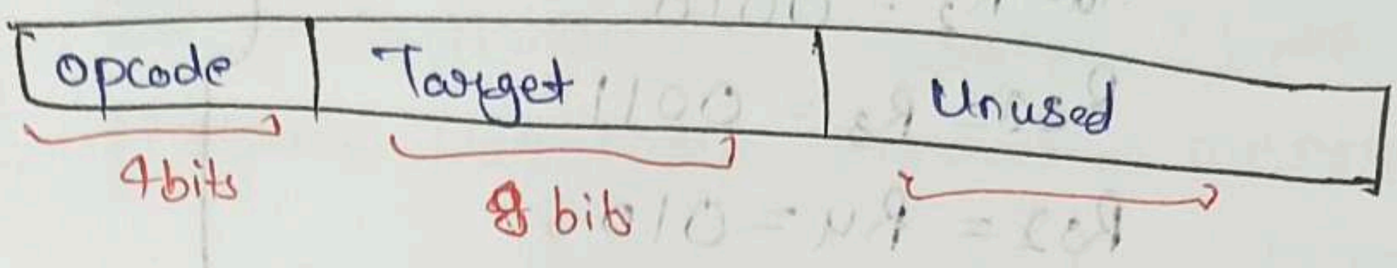
⑥ Absolute Jump

JMP: Target

It uses an absolute byte address.

0 to 255, 2⁸

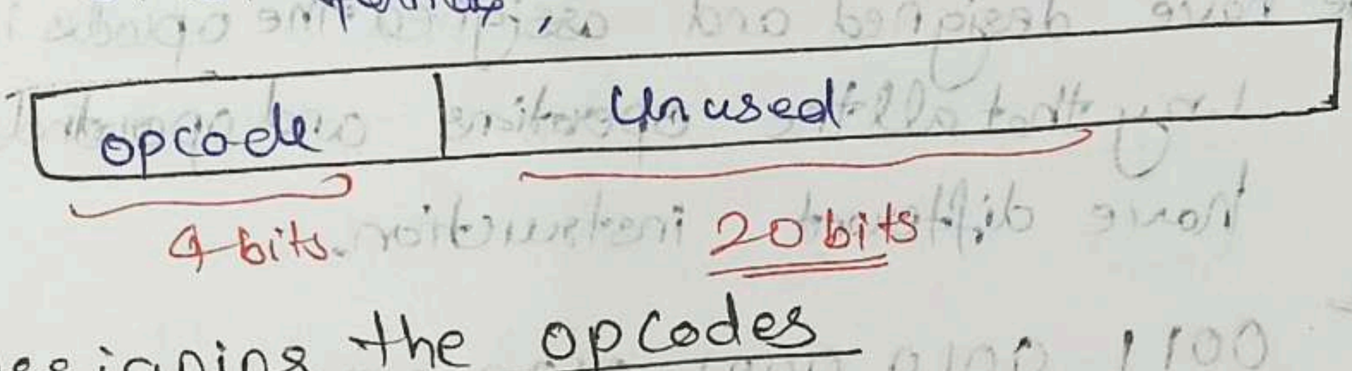
To represent it, 8 bits are required.



No: of unused bits = 12.

⑦ Nop (No operation will be performed)

Therefore only opcode is enough to represent it, but to make instructions length equal to that of all other formats,



* Assigning the opcodes

~~There is only~~ ...

We have 16 diff opcodes and each ~~contain~~ should be represented by 4 bits

then we can arrange them in (16!) ways

One such possible assignment of opcode is

Instruction	opcode	Instruction	Opcode
ADD	0000	LDI	1000
SUB	0001	LD	1001
AND	0010	ST	1010
OR	0011	BEQ	1011
XOR	0100	BNE	1100
INC	0101	JMP	1101
DEC	0110	RET	1110
MOV	0111	NOP	1111

Example:

Each operation has a unique opcode.

Suppose we want to do Add.

ADD, R_2, R_3, R_4

Opcode AND = 0000

$R_d = R_2$ = 0010

$R_{S1} = R_3$ = 0011

$R_{S2} = R_4$ = 0100

Unused bits = 00000000

Instruction encoding

0000 0010 0011 0100 0000 0000

opcode R_d R_{S_1} R_{S_2} unused

Note:- We have designed and assigned the opcodes in such a way that all the operations and operands have different instruction.

Example

0011 0010 0001 0100 00000000

opcode unused bits.

opcode "0011" ⇒ OR operation

R_d = 0010

R_{S_1} = 0001

R_{S_2} = 0100

Example

1101 0010 1000 0000 0000 0000

↳ This opcode indicates Jump operation

Target = 00101000 (40)

Unused bits = 12 zeroes

"Jmp 40"

→ Our 24 bit instruction length is better than 16-bit design because it fits all the required fields using straightforward, consistent opcode decoding.

But each instruction occupies 3 bytes, so programs use more instruction memory.